UNIVERSIDADE DO VALE DO ITAJAÍ

Curso de Ciência da Computação

SIMULADOR DE ESCALONAMENTO MULTIPROCESSADO COM PAGINAÇÃO SOB DEMANDA

José Gabriel Santos Gomes

Matheus Pompeo

Nathan Gustavo Padilha Reichert

Disciplina: Sistemas Operacionais

Professor: Michael Douglas Cabral Alves

Semestre: 2026/2

Itajaí (SC)

2026

RESUMO

Este trabalho apresenta um simulador de sistema operacional simplificado que integra **escalonamento de CPU** e **paginação sob demanda**. O simulador recebe cargas de 1000 processos em formato JSON, cada um alternando trechos de CPU e de E/S e referenciando páginas a cada unidade de CPU executada, e permite escolher, por argumentos de linha de comando, o escalonador (FCFS, SJF não preemptivo ou Round Robin com quantum variável), o número de CPUs, a política de substituição de páginas (FIFO ou LRU), a quantidade de quadros e o custo da troca de contexto. A implementação, em C++17, foi validada contra todos os exemplos numéricos do enunciado por meio de testes automáticos. Foram realizadas 112 execuções, entre elas as 28 execuções base exigidas. Os resultados mostram que, neste modelo, a memória domina o desempenho: cerca de metade das referências gera page fault com 256 quadros, a troca de contexto chega a ocupar 37% do tempo de CPU e o speedup com 4 CPUs chega a ser superlinear (4,85) porque a máquina maior sofre menos faults. Também foram observados efeito comboio no FCFS, adiamento dos processos longos no SJF e thrashing com poucos quadros.

Palavras-chave: escalonamento de CPU; memória virtual; paginação sob demanda; substituição de páginas; multiprocessamento; simulação.

1 INTRODUÇÃO

Escalonamento de CPU e gerenciamento de memória costumam ser estudados em capítulos separados, mas em um sistema real eles interagem o tempo todo: um processo que sofre page fault sai da CPU, e a escolha de quem executa muda quais páginas permanecem na memória (SILBERSCHATZ; GALVIN; GAGNE, 2018). O objetivo deste trabalho é construir um simulador que modele essa interação e, com dados coletados por nós, analisar o comportamento dos algoritmos em diferentes cenários de carga e de hardware.

1.1 Enunciado

O enunciado pede um simulador que receba uma carga de 1000 processos e permita observar, por meio de métricas, como diferentes políticas de escalonamento e de memória se comportam em máquinas com 1 e com 4 CPUs. Os objetivos específicos são:

- implementar os escalonadores FCFS, SJF (não preemptivo) e Round Robin com quantum variável;
- simular máquinas com 1 e com 4 CPUs compartilhando uma única fila de prontos;
- simular memória física paginada com page fault e substituição FIFO e LRU;
- modelar o custo da troca de contexto e entender seu efeito no desempenho;
- acompanhar e registrar os conjuntos de processos (prontos, executando, bloqueados, terminados);
- coletar, calcular e interpretar métricas de desempenho, justiça e uso de recursos;
- produzir o diagrama de Gantt da janela de tempo de 0 a 200;
- investigar fenômenos como injustiça, starvation, efeito comboio e thrashing;
- justificar as conclusões com dados e teoria e reconhecer as limitações do modelo.

Cada grupo recebeu dois arquivos: `cenario_A_t0.json`, com os 1000 processos chegando em t = 0, e `cenario_B_espacadas.json`, com chegadas espalhadas no tempo e carga média próxima de 95% da capacidade de 4 CPUs. A Tabela 1 resume as características da carga, extraídas dos próprios arquivos.

Tabela 1: Características da carga (iguais nos cenários A e B)

Classe	Processos	CPU total média	Bursts de CPU (média)	Burst médio	Maior burst	Páginas (média / máx.)
curto	947	16,4	3,0	5,5	10	6,0 / 8
longo	53	500,4	2,6	195,0	299	22,7 / 32

Fonte: elaborado pelos autores (2026).

A carga é bimodal: os 53 processos longos (5,3% do total) concentram 26.522 das 42.050 unidades de CPU (63%). As 6.890 páginas existentes somam 27 vezes os 256 quadros padrão, o que já indica que a memória será um recurso disputado. O repositório com o código, os testes e todos os resultados está em https://github.com/naasdd/trabalho-m2-so.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Processos, estados e filas

Um **processo** é um programa em execução. Durante sua vida ele alterna **bursts de CPU**, trechos em que precisa do processador, e trechos de **E/S**, em que espera um dispositivo e não usa CPU. O sistema operacional mantém cada processo em um estado: novo, pronto, executando, bloqueado ou terminado. Os processos prontos aguardam na **fila de prontos**, e é nela que se mede o tempo de espera (TANENBAUM; BOS, 2016).

2.2 Escalonadores

O **FCFS** (*First Come, First Served*) atende os processos na ordem de chegada à fila. É simples e justo na ordem, mas sofre o **efeito comboio**: um processo com burst longo segura todos os curtos atrás dele. O **SJF** (*Shortest Job First*) escolhe o processo com o menor próximo burst de CPU; para um conjunto fixo de processos ele minimiza a espera média, mas exige conhecer a duração dos bursts e pode adiar indefinidamente os processos longos (**starvation**). O **Round Robin** (RR) dá a cada processo uma fatia de tempo, o **quantum**; quando ela acaba e há outro processo pronto, o atual volta para o fim da fila. Quantum pequeno melhora o tempo de resposta mas aumenta o número de trocas de contexto; quantum muito grande faz o RR se comportar como o FCFS (SILBERSCHATZ; GALVIN; GAGNE, 2018).

2.3 Troca de contexto e multiprocessamento

A **troca de contexto** salva o estado do processo que sai da CPU e carrega o do que entra. Durante esse tempo a CPU está ocupada, mas nenhum processo avança, então trocas são custo puro. Com várias CPUs e uma única fila global, os núcleos extras permitem que processos diferentes executem ao mesmo tempo. O ganho é medido pelo **speedup** (makespan com 1 CPU dividido pelo makespan com 4 CPUs) e pela **eficiência** (speedup dividido por 4).

2.4 Paginação sob demanda e substituição

Na paginação, o processo é dividido em **páginas** e a memória física em **quadros** do mesmo tamanho. Na paginação sob demanda uma página só é trazida do disco quando é referenciada; se ela não está em nenhum quadro ocorre um **page fault** e o processo fica bloqueado até a página chegar. Quando a memória está cheia, uma página **vítima** precisa sair. O **FIFO** retira a página carregada há mais tempo, e o **LRU** (*Least Recently Used*) retira a página referenciada há mais tempo, aproveitando a **localidade de referência** (BELADY, 1966). Se a memória é pequena demais para o conjunto de páginas em uso, o sistema passa mais tempo trazendo páginas do que executando, fenômeno chamado **thrashing** (DENNING, 1968).

3 O SIMULADOR

3.1 Visão geral e tecnologias

O simulador foi escrito em **C++17** sem nenhuma chamada específica de sistema operacional, então compila igual no macOS, no Linux e no Windows (MinGW). A leitura do JSON usa a biblioteca de arquivo único nlohmann/json, indicada no enunciado e incluída no repositório, de modo que não é preciso instalar nada além do compilador. Os scripts de testes, de experimentos e de gráficos são em Python 3. O código foi dividido em cinco módulos (Tabela 2).

Tabela 2: Organização do código

Arquivo	Responsabilidade
src/main.cpp	Lê e valida os argumentos da linha de comando
src/processo.h/.cpp	Structs Processo e Burst e leitura do arquivo JSON
src/memoria.h/.cpp	Quadros, busca de página, escolha de vítima (FIFO/LRU) e proteção
src/simulador.h/.cpp	Loop do relógio, escalonadores, despacho e page fault
src/saida.h/.cpp	Métricas, resumo, Gantt, log de estados e CSVs

Fonte: elaborado pelos autores (2026).

Todos os parâmetros são escolhidos na execução, sem recompilar: `--arquivo`, `--cpus`, `--sched` (fcfs, sjf ou rr), `--quantum`, `--subst` (fifo ou lru), `--quadros` (padrão 256), `--custo-troca` (padrão 1) e `--log`. Foram acrescentados `--custo-fault` (padrão 20) e `--sem-memoria`, necessários para reproduzir os exemplos 7.1 e 7.2 do enunciado, que usam fault de 5 unidades e nenhuma memória.

3.2 Modelo de tempo e o loop principal

O tempo é discreto. O simulador mantém um relógio que avança de uma em uma unidade e, a cada unidade, executa cinco passos que seguem exatamente a ordem de eventos da seção 6.2 do enunciado (Código 1). Essa abordagem foi escolhida por ser direta de conferir contra as regras; com cerca de 42 mil unidades de CPU na carga, cada execução de 1000 processos leva menos de 0,4 segundo.

Código 1: Loop principal (src/simulador.cpp)

```
void simular(Simulacao &sim)
{
    int total = sim.processos.size();
    while (sim.terminados < total)
    {
        tratarChegadasERetornos(sim); // 1. chegadas e retornos entram na fila (ordem de pid)
        tratarFimDoQuantum(sim);      // 2. quem estourou o quantum vai pro fim da fila
        despacharProcessos(sim);      // 3. CPUs livres pegam processo (CPU 0 primeiro)
        escreverLogDoInstante(sim);   //    registra o estado deste instante (se --log)
        executarUmTick(sim);          // 4. cada CPU anda 1 unidade de tempo
        finalizarTick(sim);           // 5. fim de burst / fim de quantum
        sim.tempo++;
    }
}
```

Fonte: elaborado pelos autores (2026).

O fim de um burst e o fim de um quantum são detectados no passo 5, mas valem para o instante seguinte. Assim, um processo que estourou o quantum só decide se sai da CPU no passo 2 do próximo instante, depois que as chegadas e os retornos daquele instante já entraram na fila, que é exatamente a ordem exigida pelo enunciado.

3.3 Escalonadores e troca de contexto

A fila de prontos é uma `deque<int>` com os índices dos processos na ordem de entrada. No FCFS e no RR o escolhido é o primeiro da fila. No SJF a fila é percorrida em busca do menor restante do burst de CPU atual; a comparação usa < estrito, então em caso de empate fica quem entrou primeiro, como pede o enunciado (Código 2). Quem volta de E/S ou de page fault entra no fim da fila. O SJF usa o valor do campo `cpu` do arquivo e nunca o campo `classe`.

Código 2: Escolha do próximo processo (src/simulador.cpp)

```
int escolherDaFila(Simulacao &sim)
{
    if (sim.config.escalonador != "sjf")
        return 0;                       // FCFS e RR: o primeiro da fila
    int melhor = 0;
    for (int pos = 1; pos < (int)sim.filaProntos.size(); pos++)
    {
        Processo &candidato = sim.processos[sim.filaProntos[pos]];
        Processo &atual = sim.processos[sim.filaProntos[melhor]];
        if (restanteDoBurst(candidato) < restanteDoBurst(atual))
            melhor = pos;               // empate: fica quem entrou primeiro
    }
    return melhor;
}
```

Fonte: elaborado pelos autores (2026).

No despacho, a CPU compara o processo com o último que ela executou (Convenção A). Se for diferente, ela fica ocupada por `--custo-troca` unidades antes de executar; se for o mesmo, como no RR com fila vazia ou na volta de uma E/S durante a qual a CPU ficou ociosa, não há custo. A espera é acumulada no despacho (`tempoEspera += tempo - entrouNaFila`), somando todos os intervalos que o processo passou na fila, e a troca de contexto não entra na espera. Cada CPU acumula tempo útil, tempo de troca e tempo ocioso; em todas as execuções a soma das três parcelas é igual ao makespan.

3.4 Memória e page fault

A memória física é um `vector<Quadro>` com `--quadros` posições. Cada quadro guarda apenas a identidade da página, o par (pid, número da página), e três informações de controle: o instante em que foi carregada (`tempoCarga`, usado pelo FIFO), o instante da última referência (`ultimoUso`, usado pelo LRU) e se está protegida. A busca é linear. Essa estrutura foi escolhida por ser a tradução direta das regras do enunciado e por ser suficiente em desempenho: com 256 quadros, cada referência custa no máximo 256 comparações, e as cerca de 42 mil referências da carga são verificadas em frações de segundo. Uma tabela hash

por processo deixaria a busca constante, mas não mudaria nenhum resultado e tornaria a escolha da vítima mais difícil de acompanhar. O Código 3 mostra a escolha do quadro.

Código 3: Escolha do quadro livre ou da vítima (src/memoria.cpp)

```
int arrumarQuadro(Memoria &memoria)
{
    for (int i = 0; i < (int)memoria.quadros.size(); i++)   // 1. quadro livre
        if (!memoria.quadros[i].ocupado)
            return i;
    int vitima = -1, menorTempo = 0;                         // 2. memoria cheia
    for (int i = 0; i < (int)memoria.quadros.size(); i++)
    {
        Quadro &quadro = memoria.quadros[i];
        if (quadro.protegido) continue;
        int tempo = (memoria.politica == "fifo") ? quadro.tempoCarga : quadro.ultimoUso;
        if (vitima == -1 || tempo < menorTempo) { vitima = i; menorTempo = tempo; }
    }
    return vitima;                    // -1 = todos protegidos (sem vitima)
}
```

Fonte: elaborado pelos autores (2026).

A cada unidade de CPU o processo referencia uma página. Antes de executar a unidade, a função `prepararCpu` confere se a página está na memória (Código 4). Em caso de acerto, o `ultimoUso` é atualizado e a proteção é removida. Em caso de page fault, a unidade não é consumida: o processo é bloqueado por 20 unidades, um quadro é reservado (livre ou vítima) e fica protegido até o processo voltar e usar a página, e a CPU fica livre no mesmo instante, podendo despachar outro processo. É isso que faz, no exemplo 7.3, os três processos sofrerem fault no mesmo instante t = 0. As páginas de um processo em E/S continuam na memória e podem virar vítimas, e no término todos os seus quadros são liberados.

Código 4: Preparação da CPU com despacho e verificação de memória (src/simulador.cpp)

```
void prepararCpu(Simulacao &sim, int c)
{
    Cpu &cpu = sim.cpus[c];
    while (true)
    {
        if (cpu.processo == -1)
        {
            if (sim.filaProntos.empty()) return;   // CPU ociosa
            despachar(sim, c);
        }
        if (cpu.trocaRestante > 0) return;         // ainda trocando contexto
        if (sim.config.semMemoria) return;
        if (acessarPagina(sim, c)) return;         // acerto: executa
        tratarPageFault(sim, c);                   // fault: CPU livre, tenta outro
    }
}
```

Fonte: elaborado pelos autores (2026).

3.5 Saídas

Ao final de cada execução o simulador imprime o resumo padronizado: parâmetros, métricas globais, tabela por CPU (útil, troca, ocioso, porcentagens e trocas) e page faults com a taxa de faults. Como extra, o resumo também separa as métricas por classe (curto e longo),

o que permite medir justiça. O Gantt da janela de 0 a 200 é salvo no formato `cpu,inicio,fim,evento`, com uma faixa por CPU. Com `--log`, o simulador registra, a cada instante em que algo muda, os eventos com o pid envolvido, a fila de prontos em ordem, os bloqueados com o instante de retorno, o processo em cada CPU e a quantidade de quadros ocupados (Código 5).

Código 5: Trecho do log de estados no exemplo 7.2 (instante 6)

```
t=6
  evento: RETORNO DE FAULT P2
  evento: PAGE FAULT P1 pag 1 -> quadro 3 (volta em 11)
  evento: DESPACHO P2 -> CPU0
  prontos:    []
  bloqueados: {P1(fault ate 11) P3(fault ate 7)}
  cpus:       CPU0=P2
  quadros ocupados: 4/256 | terminados: 0/3
```

Fonte: elaborado pelos autores (2026).

3.6 Suposições

Alguns pontos do enunciado admitem mais de uma leitura. As decisões tomadas estão na Tabela 3.

Tabela 3: Suposições do modelo

Ponto	Decisão adotada
Quantums do RR	A tabela da seção 11 cita 2, 10 e 50, mas a contagem de 28 execuções exige 5 quantums. Usamos 2, 5, 10, 20 e 50
Momento da reserva do quadro	No instante do page fault, já protegido até o processo usar a página
Fault sem vítima	O processo fica bloqueado 20 unidades sem quadro e tenta de novo; ao conseguir o quadro a página é carregada (mais 20). O fault conta uma vez
Taxa de faults	Page faults divididos pelo total de referências da carga (42.050)
Quantum	Conta apenas unidades executadas; a troca de contexto não consome quantum
Preempções simultâneas	Tratadas em ordem de índice de CPU
Empate na vítima	Fica o quadro de menor índice, o que torna os resultados determinísticos
Trocas com custo 0	Não são contadas, pois a métrica é o número de vezes que o custo foi cobrado

Fonte: elaborado pelos autores (2026).

3.7 Validação

Antes de usar os arquivos de 1000 processos, o simulador foi validado contra todos os exemplos do enunciado, inclusive os exercícios de refazer o exemplo 7.1 com SJF e com RR de quantum 2, calculados à mão. Também foram criados casos próprios para regras que os exemplos não cobrem. O script `testes/rodar_testes.py` executa os 15 casos e compara espera, resposta, turnaround, término, page faults, makespan, as parcelas de cada CPU e o

Gantt com os valores esperados (Tabela 4). Todos passam. Para garantir que os testes realmente detectam erros, seis regras foram alteradas de propósito (desempate do SJF, preempção do RR, proteção de página, ordem dos eventos, FIFO trocado por LRU e troca de contexto) e cada alteração fez ao menos um teste falhar.

Tabela 4: Testes de validação

Teste	Configuração	Resultado esperado e obtido
7.1 FCFS	1 CPU, troca 1, sem memória	espera 0/6/8, turnaround 6/10/11, útil 10, troca 3, makespan 13
7.1 SJF	idem	espera 7/0/2, turnaround 13/4/5, makespan 13
7.1 RR q=2	idem	espera 8/9/4, resposta 0/3/4, 6 trocas, makespan 16
7.2	FCFS, troca 0, fault 5	término 12/13/8, espera 0/0/0, 5 faults
7.3 FCFS e SJF	troca 0, fault 20	espera média 3,67 (FCFS) e 2,33 (SJF)
5.4 e 5.5	processo 17 sozinho	turnaround 19 sem faults e 99 com 4 faults
4 CPUs	5 processos, troca 1	P5 na CPU 0 em t=4, CPU 0: útil 6, troca 2
FIFO e LRU	refs 0 1 2 0 3 0 4, 3 quadros	6 faults (FIFO) e 5 faults (LRU)
Sem vítima	1 quadro, 2 processos	P2 tenta em 5, carrega em 10 e termina em 16
Regras extras	empate SJF, RR sozinho, volta de E/S	desempate pela fila, sem troca extra

Fonte: elaborado pelos autores (2026).

4 METODOLOGIA EXPERIMENTAL

Todas as execuções foram feitas pelo script `scripts/experimentos.py`, que chama o simulador com cada combinação de parâmetros e grava os resultados em CSV, e os gráficos foram gerados por `scripts/graficos.py`. Como o simulador é determinístico, rodar os scripts de novo reproduz exatamente os mesmos números. O conjunto completo levou cerca de 15 segundos em um MacBook. Foram feitas 112 execuções:

- **28 execuções base**: cenários A e B, 1 e 4 CPUs, escalonadores FCFS, SJF e RR com quantum 2, 5, 10, 20 e 50, todas com LRU, 256 quadros e custo de troca 1;
- **FIFO × LRU** em 8 configurações (o mínimo exigido é 4): cenários A e B, 1 e 4 CPUs, FCFS e RR com quantum 10;
- **variação de quadros** (opcional): 32, 64, 128, 256, 512, 1024 e 2048 quadros, com RR de quantum 10 e LRU;

- **variação do custo de troca**: 0, 1, 2 e 5, com 4 CPUs, FCFS e RR com quantum 2 e 10.

No texto, cada execução é identificada por cenário, número de CPUs e escalonador. Por exemplo, **B-4-RR10** é o cenário B com 4 CPUs e Round Robin de quantum 10, nas condições base.

5 RESULTADOS

As Tabelas 5 a 8 trazem as 28 execuções base, uma linha por execução e uma coluna por métrica obrigatória. Para caber na página, cada cenário foi dividido em duas tabelas: uma com as métricas globais e outra com as métricas por CPU, que listam os valores das CPUs 0, 1, 2 e 3 separados por barra.

Tabela 5: Cenário A (todos em t = 0), métricas globais (LRU, 256 quadros, troca 1)

CPUs	Escal.	Espera média	Espera máx.	Turnaround médio	Resposta média	Makespan	Speedup	Efic.	Trocas	Page faults	Taxa faults (%)
1	FCFS	11504,8	59.803	25300,7	499,5	66.691	3,93	0,98	24.534	21.580	51,3
1	SJF	14706,0	51.937	14941,0	7533,2	52.878	3,85	0,96	10.360	7.421	17,6
1	RR2	12287,6	71.053	25987,1	499,5	77.860	3,97	0,99	35.788	21.371	50,8
1	RR5	11654,5	63.196	25453,8	499,5	69.721	3,96	0,99	27.633	21.595	51,4
1	RR10	11564,9	60.952	25367,8	499,5	67.862	3,94	0,98	25.737	21.872	52,0
1	RR20	11515,9	59.925	25312,2	499,5	66.833	3,94	0,98	24.724	21.597	51,4
1	RR50	11505,0	59.789	25301,2	499,5	66.682	3,94	0,98	24.563	21.592	51,3
4	FCFS	2483,1	11.639	6419,0	124,5	16.968	3,93	0,98	24.603	21.653	51,5
4	SJF	3633,9	12.688	3894,4	1855,4	13.748	3,85	0,96	11.586	8.633	20,5
4	RR2	2655,6	14.117	6585,0	124,5	19.610	3,97	0,99	35.696	21.540	51,2
4	RR5	2515,2	12.180	6455,8	124,5	17.583	3,96	0,99	27.701	21.734	51,7
4	RR10	2491,8	11.712	6432,9	124,5	17.227	3,94	0,98	25.695	21.858	52,0
4	RR20	2483,9	11.607	6420,6	124,5	16.980	3,94	0,98	24.787	21.684	51,6
4	RR50	2483,1	11.613	6419,1	124,5	16.923	3,94	0,98	24.619	21.659	51,5

Fonte: elaborado pelos autores (2026).

Tabela 6: Cenário A (todos em t = 0), métricas por CPU

CPUs	Escal.	Trocas por CPU	Utilização por CPU (%)	Ocioso por CPU
1	FCFS	24534	99,8	107
1	SJF	10360	99,1	468
1	RR2	35788	100,0	22
1	RR5	27633	99,9	38
1	RR10	25737	99,9	75
1	RR20	24724	99,9	59
1	RR50	24563	99,9	69
4	FCFS	6155/6137/6168/6143	99,2 / 98,5 / 97,7 / 97,5	139/256/395/429
4	SJF	2781/2854/2893/3058	98,3 / 99,5 / 96,8 / 95,5	233/63/442/618
4	RR2	8919/8963/8888/8926	99,2 / 99,6 / 98,9 / 98,8	163/76/221/234
4	RR5	6906/6920/6939/6936	99,6 / 99,4 / 98,9 / 98,8	64/113/195/209
4	RR10	6409/6398/6470/6418	98,9 / 98,6 / 98,0 / 97,7	183/246/346/388
4	RR20	6162/6165/6206/6254	99,2 / 98,9 / 97,8 / 97,7	133/191/371/388
4	RR50	6140/6162/6193/6124	99,3 / 98,7 / 98,2 / 97,8	124/212/307/380

Fonte: elaborado pelos autores (2026).

Tabela 7: Cenário B (chegadas espaçadas), métricas globais (LRU, 256 quadros, troca 1)

CPUs	Escal.	Espera média	Espera máx.	Turnaround médio	Resposta média	Makespan	Speedup	Efic.	Trocas	Page faults	Taxa faults (%)
1	FCFS	10819,0	59.462	18814,6	479,5	65.795	4,85	1,21	23.643	20.688	49,2
1	SJF	8532,2	49.026	8755,0	3586,8	52.133	3,93	0,98	9.787	6.842	16,3
1	RR2	11854,4	70.772	20203,9	478,5	77.622	4,21	1,05	35.538	20.731	49,3
1	RR5	11003,3	62.357	19052,2	480,5	68.933	4,59	1,15	26.863	20.790	49,4
1	RR10	10869,7	60.699	18889,2	479,1	66.963	4,78	1,20	24.886	21.002	49,9
1	RR20	10819,6	59.292	18906,1	480,5	65.939	4,78	1,20	23.846	20.712	49,3
1	RR50	10822,2	59.459	18818,1	479,5	65.795	4,84	1,21	23.672	20.702	49,2
4	FCFS	414,8	4.994	658,3	38,3	13.567	4,85	1,21	10.778	7.826	18,6
4	SJF	129,5	7.259	327,2	23,4	13.280	3,93	0,98	8.592	5.649	13,4
4	RR2	833,7	11.119	1197,0	29,0	18.454	4,21	1,05	30.903	12.807	30,5
4	RR5	473,8	6.561	757,3	29,1	15.003	4,59	1,15	16.677	9.533	22,7
4	RR10	414,6	5.363	674,0	31,8	14.006	4,78	1,20	12.868	8.519	20,3
4	RR20	416,0	5.202	670,5	34,5	13.786	4,78	1,20	11.723	8.329	19,8
4	RR50	412,9	5.240	660,9	36,7	13.596	4,84	1,21	11.085	8.038	19,1

Fonte: elaborado pelos autores (2026).

Tabela 8: Cenário B (chegadas espaçadas), métricas por CPU

CPUs	Escal.	Trocas por CPU	Utilização por CPU (%)	Ocioso por CPU
1	FCFS	23643	99,8	102
1	SJF	9787	99,4	296
1	RR2	35538	100,0	34
1	RR5	26863	100,0	20
1	RR10	24886	100,0	27
1	RR20	23846	99,9	43
1	RR50	23672	99,9	73
4	FCFS	2613/2710/2752/2703	98,3 / 98,7 / 95,8 / 96,5	235/171/565/469
4	SJF	2394/2226/2091/1881	97,8 / 96,6 / 95,2 / 91,7	292/450/633/1103
4	RR2	7723/7755/7718/7707	99,8 / 98,8 / 98,4 / 98,3	39/220/297/307
4	RR5	4220/4160/4135/4162	99,1 / 98,1 / 97,2 / 97,2	142/289/427/427
4	RR10	3244/3233/3228/3163	99,4 / 98,5 / 97,8 / 96,4	83/214/305/504
4	RR20	2972/2878/2970/2903	99,3 / 98,0 / 97,0 / 95,9	101/280/420/570
4	RR50	2737/2677/2905/2766	98,4 / 99,1 / 97,0 / 96,4	224/125/407/493

Fonte: elaborado pelos autores (2026).

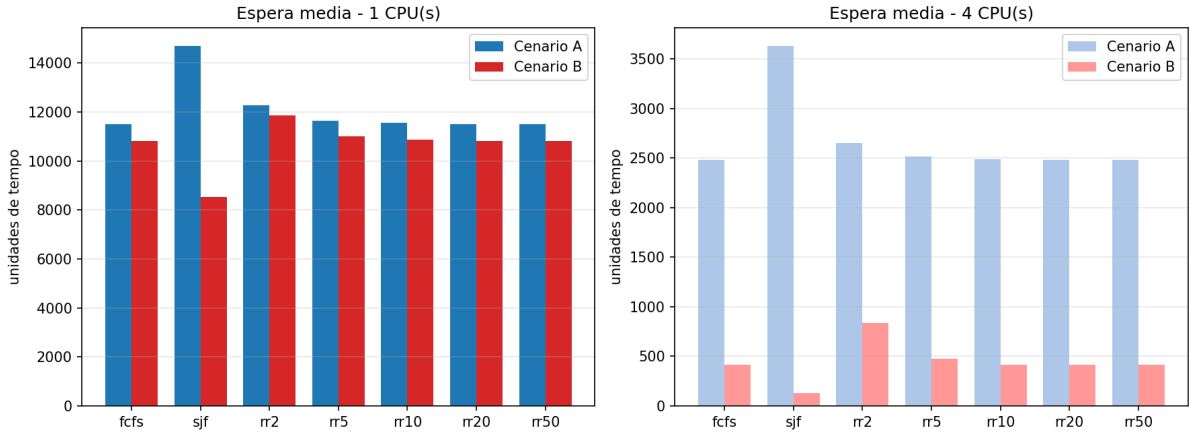
A Tabela 9 mostra o speedup e a eficiência, também representados na Figura 3. As Figuras 1 a 7 resumem os demais experimentos.

Tabela 9: Speedup e eficiência por escalonador

Escal.	A: makespan 1 CPU	A: makespan 4 CPUs	A: speedup	A: efic.	B: makespan 1 CPU	B: makespan 4 CPUs	B: speedup	B: efic.
FCFS	66.691	16.968	3,93	0,98	65.795	13.567	4,85	1,21
SJF	52.878	13.748	3,85	0,96	52.133	13.280	3,93	0,98
RR2	77.860	19.610	3,97	0,99	77.622	18.454	4,21	1,05
RR5	69.721	17.583	3,96	0,99	68.933	15.003	4,59	1,15
RR10	67.862	17.227	3,94	0,98	66.963	14.006	4,78	1,20
RR20	66.833	16.980	3,94	0,98	65.939	13.786	4,78	1,20
RR50	66.682	16.923	3,94	0,98	65.795	13.596	4,84	1,21

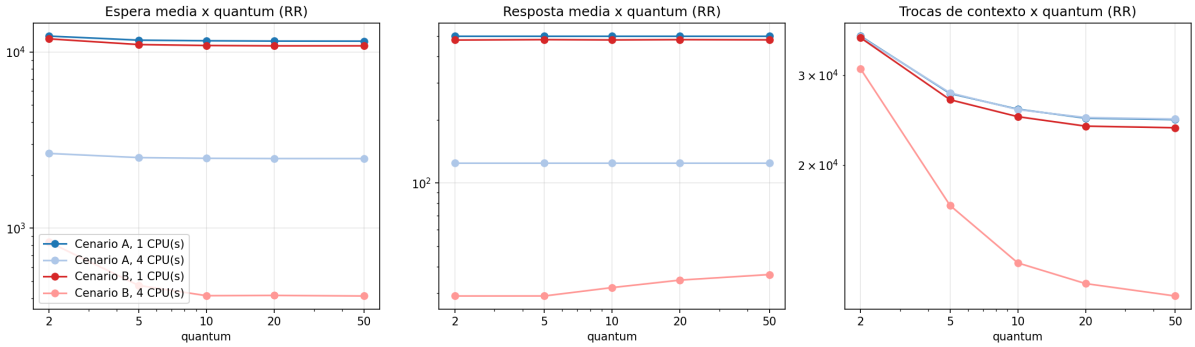
Fonte: elaborado pelos autores (2026).

Figura 1: Espera média por escalonador



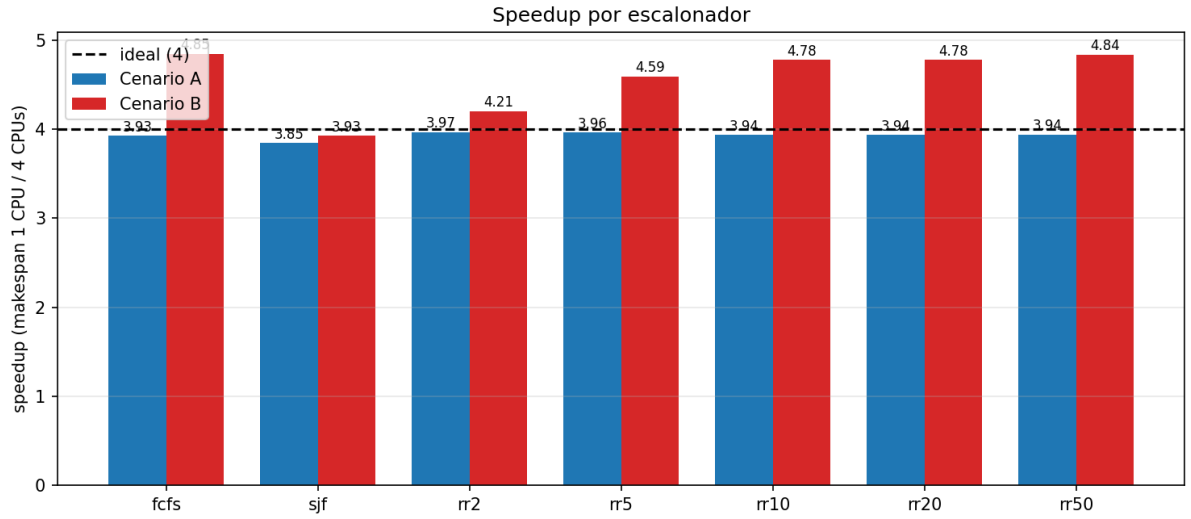
Fonte: elaborado pelos autores (2026).

Figura 2: Efeito do quantum no RR (eixos em escala logarítmica)



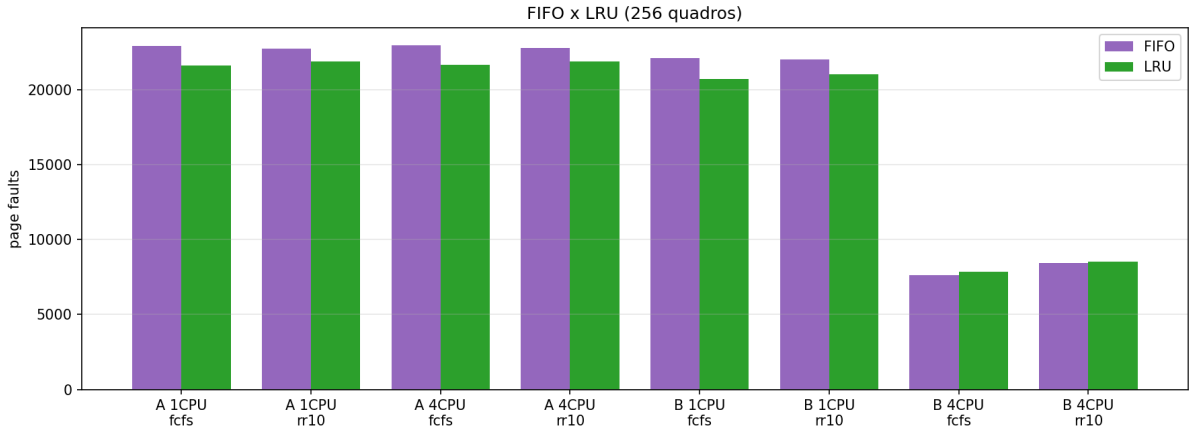
Fonte: elaborado pelos autores (2026).

Figura 3: Speedup por escalonador



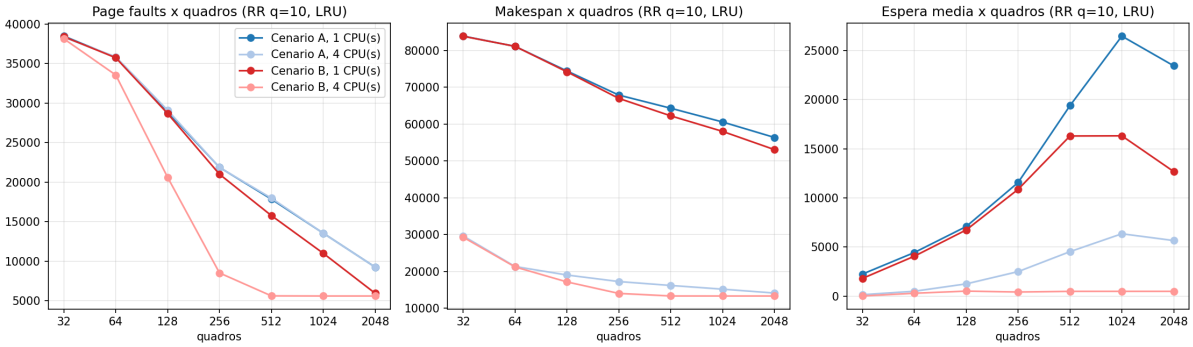
Fonte: elaborado pelos autores (2026).

Figura 4: Page faults com FIFO e com LRU



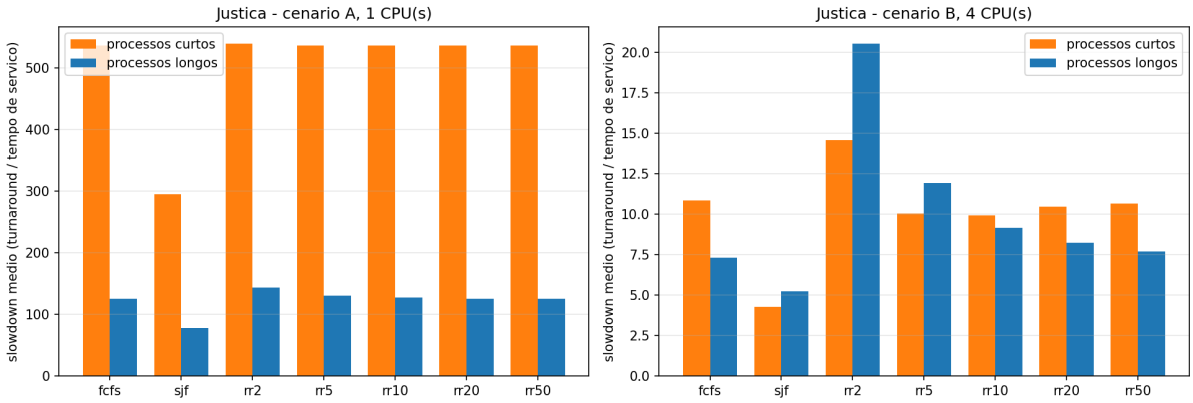
Fonte: elaborado pelos autores (2026).

Figura 5: Efeito da quantidade de quadros



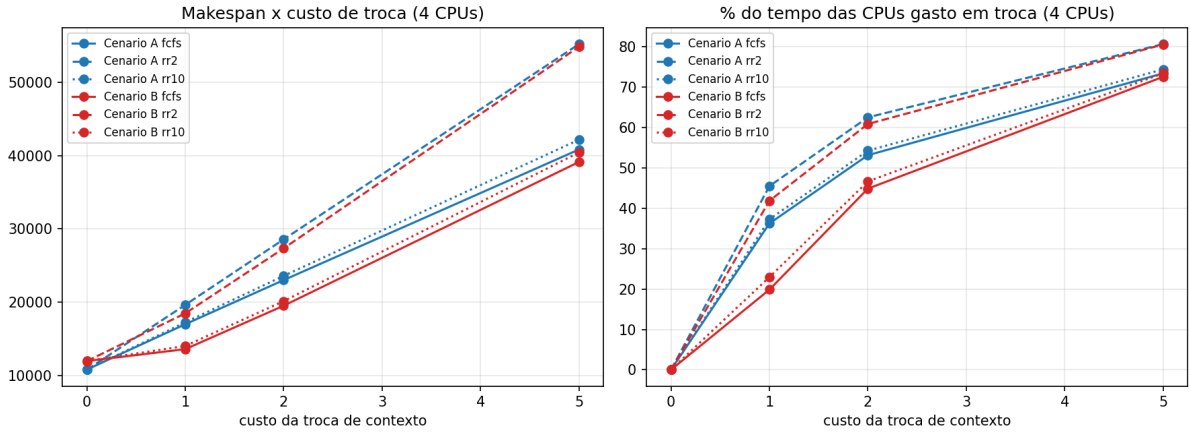
Fonte: elaborado pelos autores (2026).

Figura 6: Slowdown médio de processos curtos e longos



Fonte: elaborado pelos autores (2026).

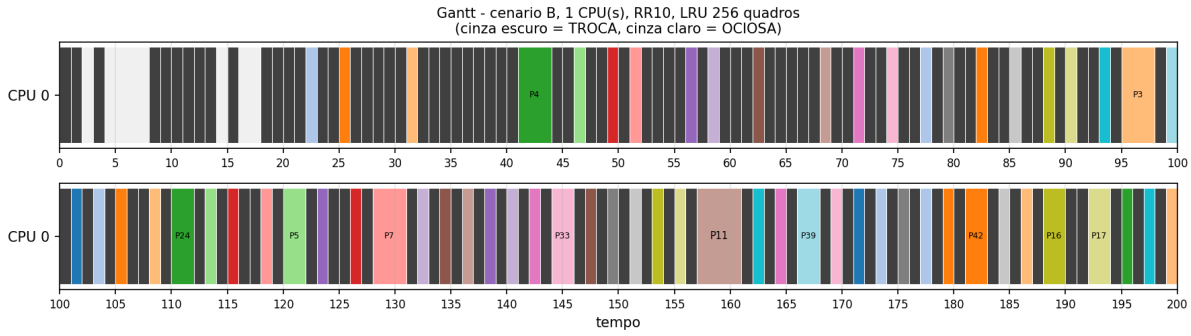
Figura 7: Efeito do custo da troca de contexto



Fonte: elaborado pelos autores (2026).

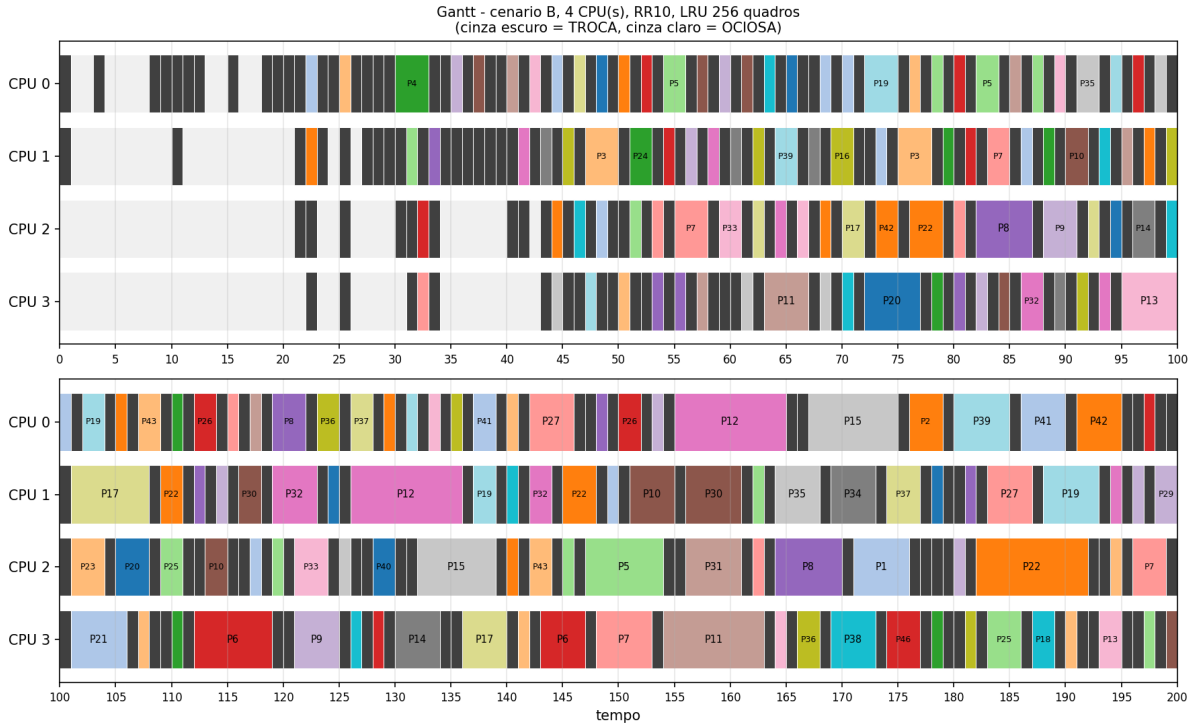
As Figuras 8 e 9 trazem o diagrama de Gantt da janela de 0 a 200 para o cenário B com RR de quantum 10, com 1 e com 4 CPUs. Os Gantt de todas as 28 execuções base estão em `resultados/gantt/` no repositório.

Figura 8: Gantt de 0 a 200, B-1-RR10



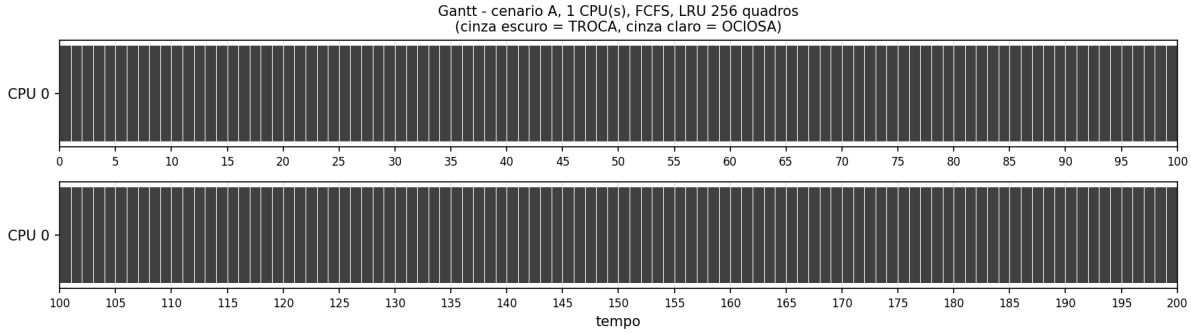
Fonte: elaborado pelos autores (2026).

Figura 9: Gantt de 0 a 200, B-4-RR10



Fonte: elaborado pelos autores (2026).

Figura 10: Gantt de 0 a 200, A-1-FCFS (a CPU só troca contexto: cada processo despachado sofre page fault)



Fonte: elaborado pelos autores (2026).

6 ANÁLISE E DISCUSSÃO

6.1 A memória domina o desempenho

O primeiro resultado que chama atenção é o peso da memória. Com 256 quadros, a taxa de faults fica em torno de 50% nas execuções com 1 CPU (A-1-FCFS: 51,3%, 21.580 faults em 42.050 referências) e só cai para cerca de 20% em B-4. Cada fault tira o processo da CPU e, quando ele volta, exige um novo despacho e uma nova troca de contexto. Por isso o número de trocas acompanha o número de faults: A-1-FCFS tem 24.534 trocas, que ocupam 36,8% do makespan de 66.691. A Figura 10 mostra o caso extremo: em toda a janela de 0 a 200 de A-1-FCFS a CPU só faz trocas de contexto, porque cada processo é despachado, paga a troca e sofre page fault já na primeira referência.

No cenário A essa avalanche inicial também esgota a memória. Como cada fault reserva e protege um quadro, depois de 256 faults todos os quadros estão protegidos e novos faults ficam sem vítima. No log de A-1-FCFS isso aconteceu 13.842 vezes. Esse é o comportamento que motiva a regra de proteção do enunciado: sem ela, uma página recém carregada poderia ser expulsa antes de ser usada.

6.2 Escalonadores, efeito comboio e o caso contraintuitivo do SJF

No cenário B com 4 CPUs, o SJF teve a menor espera média (B-4-SJF: 129,49, contra 414,80 em B-4-FCFS e 414,58 em B-4-RR10), como prevê a teoria. A diferença vem quase toda dos processos curtos: a espera média deles cai de 323,99 no FCFS para 41,87 no SJF. Esse é o **efeito comboio**: no FCFS um curto fica atrás de longos, cujo burst médio é de 195 unidades contra 5,5 dos curtos (Tabela 1).

Com 1 CPU no cenário A o resultado parece contradizer a teoria: a espera média do SJF (14.706,01) é maior que a do FCFS (11.504,81). Porém o turnaround médio do SJF é 41% menor (14.940,98 contra 25.300,68), e o SJF sofreu 2,9 vezes menos faults (7.421 contra 21.580). A explicação está na definição da espera, que só conta o tempo na fila de prontos. No FCFS os processos passam muito tempo bloqueados em faults, tempo que entra no turnaround mas não na espera. O SJF sofre menos faults porque um processo que volta de um fault tem um restante de burst menor e por isso é escolhido logo, enquanto suas páginas ainda estão na

memória. No FCFS ele vai para o fim de uma fila de centenas de processos e, quando volta à CPU, suas outras páginas já foram expulsas. Ou seja, neste modelo o escalonador também decide a localidade de referência, e olhar só a espera levaria a uma conclusão errada.

6.3 Starvation e injustiça

Nas cinco execuções examinadas processo a processo (A-1-FCFS, A-1-SJF, B-4-FCFS, B-4-SJF e B-4-RR10), os 20 processos com maior espera são todos longos (arquivos em `resultados/processos/`). O SJF agrava isso: em A-1-SJF o tempo de resposta médio dos longos é 31.862,38, contra 6.171,55 dos curtos, e o processo 988 esperou 51.937 unidades na fila. Em B-4-SJF a resposta média dos longos (311,85) é 43 vezes a dos curtos (7,21). Como a carga é finita, os longos acabam executando, mas o padrão é o do **starvation**: com chegadas contínuas de processos curtos eles poderiam ser adiados indefinidamente.

A justiça muda quando se mede o slowdown, a razão entre o turnaround e o tempo de serviço (Figura 6). No FCFS, os curtos são os prejudicados: em B-4-FCFS o slowdown dos curtos é 10,84 contra 7,30 dos longos, porque qualquer espera pesa muito sobre um processo de 16 unidades. O SJF inverte a situação (B-4-SJF: 4,25 para curtos e 5,24 para longos). O RR com quantum pequeno prejudica os longos: em B-4-RR2 a espera média dos longos sobe para 7.372,11, contra 2.037,45 no FCFS, porque cada quantum termina com uma volta ao fim da fila.

6.4 Efeito do quantum

Com 4 CPUs no cenário B, aumentar o quantum de 2 para 50 reduz as trocas de 30.903 para 11.085, o makespan de 18.454 para 13.596 e a espera média de 833,74 para 412,89, mas aumenta o tempo de resposta de 29,04 para 36,73 (Figura 2). É o compromisso clássico do RR: quantum pequeno responde mais rápido e gasta mais em trocas. O quantum pequeno também aumentou os faults (12.807 em B-4-RR2 contra 8.038 em B-4-RR50), porque intercalar mais processos espalha as referências e expulsa páginas que ainda seriam usadas.

Com 1 CPU o quantum quase não importa a partir de 10. Em A-1, o tempo de resposta é 499,50 para todos os quantums, e A-1-RR50 é praticamente igual a A-1-FCFS (makespan 66.682 contra 66.691). Com metade das referências gerando fault, o processo raramente executa várias unidades seguidas: o fault interrompe o burst antes de o quantum acabar. O RR só se diferencia do FCFS com quantum 2 e 5.

6.5 Custo da troca de contexto

A Figura 7 mostra o efeito do custo da troca. Em A-4-FCFS, o makespan passa de 10.775 com custo 0 para 16.968 com custo 1 e 40.824 com custo 5. Com custo 0, o makespan fica perto do limite teórico de 42.050 / 4 = 10.512,5, então praticamente toda a perda com custo maior é troca de contexto, que chega a 73,4% do tempo das CPUs com custo 5.

No cenário B aparece um efeito de realimentação: o custo da troca aumenta também os faults (B-4-FCFS: 5.669 com custo 0, 7.826 com custo 1 e 19.766 com custo 5). Trocas mais caras deixam a fila mais longa, os processos demoram mais para voltar à CPU, suas páginas são expulsas, ocorrem mais faults e, portanto, mais trocas. Esse é um exemplo concreto da interação entre escalonamento e memória que o trabalho propõe investigar.

6.6 FIFO × LRU

O LRU gerou menos faults em 6 das 8 configurações testadas (Figura 4), por exemplo em A-1-FCFS, com 21.580 contra 22.908 (5,8% a menos). A vantagem é pequena por dois motivos. Primeiro, uma parte grande dos faults é **compulsória**: toda página precisa ser carregada ao menos uma vez, e com 1024 quadros ou mais B-4-RR10 estabiliza em 5.604 faults, que é esse piso. Segundo, a substituição é global e as referências de centenas de processos se intercalam, então a recência de uma página diz pouco sobre o próximo uso dela. Nas duas configurações menos carregadas (B-4-FCFS e B-4-RR10) o FIFO foi ligeiramente melhor (7.632 contra 7.826 e 8.413 contra 8.519), mostrando que o LRU não é ótimo, apenas uma aproximação do algoritmo ótimo de Belady (BELADY, 1966).

6.7 Thrashing

A variação de quadros (Figura 5) mostra o **thrashing**. Com 32 quadros a taxa de faults passa de 90% em todas as configurações, e em A-4-RR10 a utilização das CPUs cai para 70,33% e o makespan sobe para 29.609, contra 98,31% e 17.227 com 256 quadros. As CPUs ficam ociosas não por falta de trabalho, mas porque quase todos os processos estão bloqueados esperando páginas, exatamente o quadro descrito por Denning (1968). Aumentar a memória reduz os faults continuamente até o piso compulsório.

Esse experimento também mostra por que a espera não pode ser lida isoladamente. Em A-1-RR10, a espera média sobe de 2.243,14 com 32 quadros para 26.479,04 com 1024, enquanto o turnaround médio cai de 33.454,75 para 26.844,72. Com menos faults os processos deixam de ficar bloqueados e passam a disputar a CPU na fila de prontos: o tempo apenas muda de lugar.

6.8 Speedup e o caso superlinear

No cenário A o speedup ficou entre 3,85 (SJF) e 3,97 (RR2), com eficiência de 96% a 99% (Tabela 9). Como todos os processos chegam em t = 0, sempre há trabalho para as 4 CPUs, e a perda vem apenas do final da execução, quando restam poucos processos longos.

No cenário B o speedup do FCFS chegou a 4,85, acima do ideal de 4. Isso não é erro de medição. B-1-FCFS sofreu 20.688 faults e 23.643 trocas, enquanto B-4-FCFS sofreu 7.826 faults e 10.778 trocas. As duas máquinas fazem as mesmas 42.050 unidades úteis, mas a de 1 CPU gasta 65.693 unidades ocupadas no total e a de 4 CPUs apenas 52.828. Com 1 CPU a fila é longa, as páginas são expulsas antes do reuso e o trabalho extra cresce: 4 CPUs não apenas dividem o trabalho, como também reduzem a quantidade de trabalho. É o mesmo fenômeno de speedup superlinear observado em sistemas reais quando o conjunto de trabalho passa a caber na memória ou na cache. O SJF, que já tem poucos faults com 1 CPU (6.842), ficou com speedup normal de 3,93. No cenário B o makespan com 4 CPUs também é limitado pela última chegada (t = 11.029): B-4-SJF termina apenas 2.251 unidades depois dela.

A distribuição entre as CPUs é equilibrada, mas reflete a regra de despacho pelo menor índice. Em B-4-FCFS, as CPUs 0 e 1 ficaram ociosas 235 e 171 unidades, e as CPUs 2 e 3, 565 e 469: quando sobra pouco trabalho, as CPUs de índice menor pegam primeiro.

7 LIMITAÇÕES DO MODELO

- O SJF usa a duração exata do próximo burst (campo `cpu`). Em um sistema real esse valor é desconhecido e precisaria ser estimado, por exemplo por média exponencial dos bursts anteriores.
- O tempo de page fault é fixo (20) e não existe fila de disco: todos os faults são atendidos em paralelo. Na prática o disco seria um gargalo, e o thrashing seria ainda pior.
- A troca de contexto tem custo fixo e não considera efeitos de cache ou de TLB. Também não há afinidade de processador.
- A substituição é sempre global. Políticas locais ou baseadas em conjunto de trabalho poderiam evitar que um processo roube quadros dos outros.
- Os escalonadores não têm prioridade nem envelhecimento (*aging*), então nada impede o adiamento dos longos no SJF.
- Cada configuração foi executada uma vez, com uma única carga (a seed do grupo). Como o simulador é determinístico, não há variância entre execuções, mas outras cargas poderiam gerar números diferentes.
- A interpretação da regra de fault sem vítima (Tabela 3) afeta os resultados do cenário A, em que ela ocorre com frequência.

8 CONCLUSÃO

O simulador implementa todas as regras do enunciado e reproduz todos os seus exemplos numéricos, o que foi verificado por testes automáticos. Os parâmetros podem ser trocados na execução, sem recompilar, e cada execução de 1000 processos leva menos de meio segundo.

A principal conclusão dos experimentos é que escalonamento e memória não podem ser analisados separadamente. Neste modelo a memória domina: metade das referências gera fault com 256 quadros, os faults geram trocas de contexto, e as trocas chegam a ocupar mais de um terço do tempo de CPU. O escalonador influencia a localidade, e por isso o SJF teve 2,9 vezes menos faults que o FCFS com 1 CPU. A memória também explica o speedup superlinear de 4,85 no cenário B. Os fenômenos clássicos apareceram com números: efeito comboio no FCFS, adiamento dos longos no SJF, o compromisso entre resposta e trocas no quantum do RR e thrashing com poucos quadros. Por fim, os resultados mostram que nenhuma métrica isolada basta: a espera média, sozinha, chegou a apontar o escalonador errado.

REFERÊNCIAS

ARPACI-DUSSEAU, Remzi H.; ARPACI-DUSSEAU, Andrea C. *Operating Systems: Three Easy Pieces*. Madison: Arpaci-Dusseau Books, 2018. Disponível em: https://pages.cs.wisc.edu/~remzi/OSTEP/. Acesso em: 1 nov. 2026.

BELADY, Laszlo A. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, v. 5, n. 2, p. 78-101, 1966.

DENNING, Peter J. The working set model for program behavior. *Communications of the ACM*, v. 11, n. 5, p. 323-333, 1968.

LOHMANN, Niels. *JSON for Modern C++*. Versão 3.11.3. 2023. Disponível em: https://github.com/nlohmann/json. Acesso em: 1 nov. 2026.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. *Operating System Concepts*. 10. ed. Hoboken: Wiley, 2018.

TANENBAUM, Andrew S.; BOS, Herbert. *Sistemas operacionais modernos*. 4. ed. São Paulo: Pearson, 2016.

APÊNDICE A: COMO EXECUTAR

Código e resultados: https://github.com/naasdd/trabalho-m2-so. Para compilar e rodar:

Código 6: Comandos de execução

```
make                        # compila (no Windows: mingw32-make)
make teste                  # roda os 15 testes de validacao
./simulador --arquivo cenarios/cenario_B_espacadas.json \
            --cpus 4 --sched rr --quantum 10 --subst lru --log
make experimentos           # refaz as 112 execucoes (resultados/*.csv)
make graficos               # refaz as figuras (precisa de: pip3 install matplotlib)
```

Fonte: elaborado pelos autores (2026).